ENVIRONMENTAL MONITORING DATA ANALYZER

Python Mini Project Report

Air-quality / water-quality readings using nested lists, averages, extremes, and threshold-based classification

Project Title	Environmental Monitoring Data Analyzer
Programming Language	Python 3
Data Structure	Nested lists
Report Includes	Problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, source code, test

Prepared as an educational demonstration. Thresholds are illustrative and should be replaced with applicable official standards for real monitoring.

1. Problem Statement

Environmental monitoring produces repeated air-quality or water-quality readings from different locations or sampling periods. Manually reviewing many readings can be slow and may lead to mistakes. The aim of this project is to write a Python program that stores readings in nested lists, calculates the average for each location, identifies minimum and maximum values, and classifies each location using user-supplied threshold values.

This sample project uses air-quality readings represented by an illustrative AQI-like index. The same logic can be adapted for water-quality measurements such as pH, turbidity, or dissolved oxygen by changing the measurement labels and classification rules.

2. Objectives

• Store readings for multiple monitoring locations in a nested list.

• Calculate each location's average reading.

• Find the minimum and maximum reading for each location.

• Classify averages against supplied threshold values.

• Display clear, readable results and test the program with normal and boundary inputs.

3. Assumptions

• Each inner list contains numeric readings for one location.

• All locations use the same measurement and unit in one run.

• Each location has at least one reading; empty lists are rejected.

• Readings are non-negative for this illustrative AQI-style example.

• The user supplies two thresholds: a lower threshold and an upper threshold, where lower < upper.

• Classification rules: average ≤ lower = GOOD; lower < average ≤ upper = MODERATE; average > upper = HIGH.

• The sample thresholds are for programming practice only, not official health or environmental guidance.

4. Algorithm

1. Start the program.

2. Define the nested list of readings and the two threshold values.

3. Check that the thresholds are valid and each location contains readings.

4. For each location, calculate the average, minimum, and maximum.

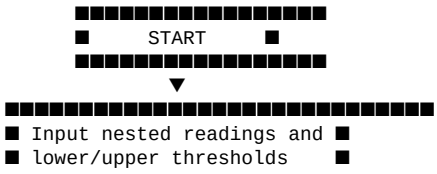
5. Compare the average with the lower and upper thresholds.

6. Assign GOOD, MODERATE, or HIGH according to the rules.

7. Display each location's readings, average, minimum, maximum, and classification.

8. Stop.

5. Flowchart



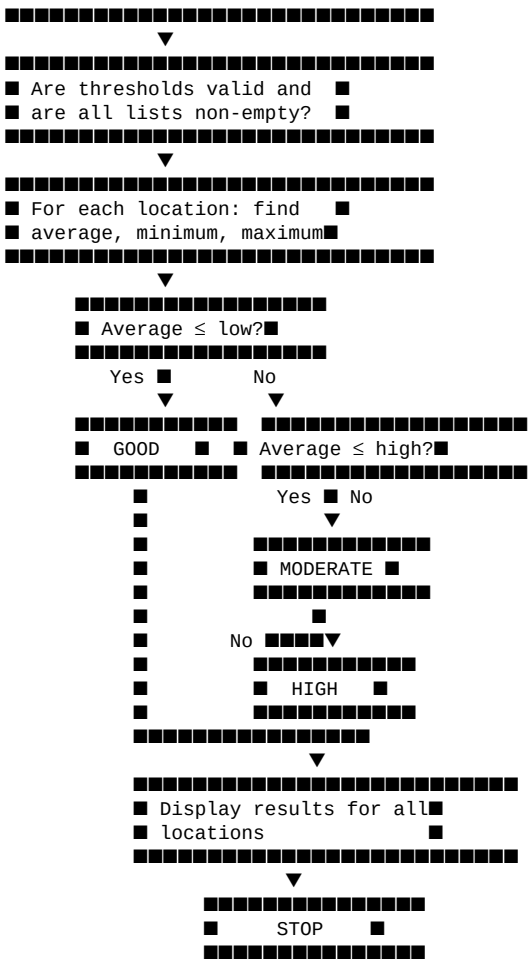


Figure 1. Program flow (text flowchart).

6. Pseudocode

```
BEGIN
    readings ← [[42, 55, 48], [72, 85, 80], [110, 125, 118]]
    lower_threshold ← 60
    upper_threshold ← 100

    IF lower_threshold >= upper_threshold:
        PRINT "Invalid thresholds"
        STOP

    FOR each location_readings in readings:
        IF location_readings is empty:
            PRINT "Invalid data"
            STOP

        average ← SUM(location_readings) / LENGTH(location_readings)
        minimum ← MIN(location_readings)
        maximum ← MAX(location_readings)

        IF average <= lower_threshold:
            category ← "GOOD"
        ELSE IF average <= upper_threshold:
            category ← "MODERATE"
        ELSE:
            category ← "HIGH"

        PRINT average, minimum, maximum, category
END
```

7. Python Source Code

```
def classify_reading(average, lower_threshold, upper_threshold):
    if average <= lower_threshold:
        return "GOOD"
    elif average <= upper_threshold:
```

```
        return "MODERATE"
    return "HIGH"


def analyze_monitoring_data(readings, lower_threshold, upper_threshold):
    if lower_threshold >= upper_threshold:
        raise ValueError("Lower threshold must be less than upper threshold.")

    if not readings:
        raise ValueError("Provide readings for at least one location.")

    results = []

    for location_number, values in enumerate(readings, start=1):
        if not values:
            raise ValueError(f"Location {location_number} has no readings.")

        if any(not isinstance(value, (int, float)) for value in values):
            raise ValueError(f"Location {location_number} contains non-numeric data.")

        average = sum(values) / len(values)
        minimum = min(values)
        maximum = max(values)
        category = classify_reading(
            average, lower_threshold, upper_threshold
        )

        results.append({
            "location": location_number,
            "readings": values,
            "average": average,
            "minimum": minimum,
            "maximum": maximum,
            "category": category
        })

    return results


# Sample data: illustrative AQI-like readings, not official guidance.
readings = [
    [42, 55, 48],
    [72, 85, 80],
    [110, 125, 118]
]
lower_threshold = 60
upper_threshold = 100

try:
    results = analyze_monitoring_data(
        readings, lower_threshold, upper_threshold
    )

    print("ENVIRONMENTAL MONITORING DATA ANALYZER")
    print(f"Thresholds: GOOD <= {lower_threshold}; "
          f"MODERATE <= {upper_threshold}; HIGH > {upper_threshold}")
    print("-" * 76)

    for item in results:
        print(f"Location {item['location']}: {item['readings']}")
        print(f"  Average: {item['average']:.2f}")
        print(f"  Minimum: {item['minimum']}")
        print(f"  Maximum: {item['maximum']}")
        print(f"  Classification: {item['category']}")
except ValueError as error:
    print("Input error:", error)
```

8. Test Cases

Test	Input readings	Thresholds	Expected result
TC1	[42, 55, 48]	60, 100	Average 48.33; min 42; max 55; GOOD
TC2	[72, 85, 80]	60, 100	Average 79.00; min 72; max 85; MODERATE
TC3	[110, 125, 118]	60, 100	Average 117.67; min 110; max 125; HIGH

Test	Input readings	Thresholds	Expected result
TC4	[60, 60]	60, 100	Average 60.00; GOOD (lower boundary)
TC5	[100, 100]	60, 100	Average 100.00; MODERATE (upper boundary)
TC6	[] for one location	60, 100	Input error: location has no readings
TC7	[10, 20]	100, 60	Input error: lower threshold must be less than upper

9. Sample Output Screenshots

The following are terminal-style output captures recreated in the report for documentation. They show the expected console output when the provided code is run; they are not photographs of a live computer screen.

```
ENVIRONMENTAL MONITORING DATA ANALYZER
Thresholds: GOOD <= 60; MODERATE <= 100; HIGH > 100
--------------------------------------------------------------------------------
Location 1: [42, 55, 48]
  Average: 48.33
  Minimum: 42
  Maximum: 55
  Classification: GOOD

Location 2: [72, 85, 80]
  Average: 79.00
  Minimum: 72
  Maximum: 85
  Classification: MODERATE

Location 3: [110, 125, 118]
  Average: 117.67
  Minimum: 110
  Maximum: 125
  Classification: HIGH
```

Screenshot 1. Successful run with three monitoring locations.

```
Input error: Lower threshold must be less than upper threshold.
```

Screenshot 2. Example validation error for invalid thresholds.

10. How to Run the Program

1. Install Python 3 from the official Python website if it is not already installed.
2. Copy the source code into a file named environmental_analyzer.py.
3. Open a terminal or command prompt in that folder.
4. Run: python environmental_analyzer.py (on some systems, use python3 environmental_analyzer.py).
5. Compare the console output with the sample output in this report.

11. Conclusion

The Environmental Monitoring Data Analyzer demonstrates how nested lists and basic Python functions can be used to process repeated environmental readings. It calculates average, minimum, and maximum values and classifies each location using supplied thresholds. Input validation helps prevent common errors such as empty readings or

incorrectly ordered thresholds. The program is a small educational prototype; real environmental decisions should use validated measurements, units, sampling procedures, and official standards appropriate to the pollutant or water-quality parameter.

12. Possible Enhancements

- Read observations from CSV or Excel files.
- Allow the user to enter thresholds and readings interactively.
- Generate graphs showing changes over time.
- Add date, time, sensor ID, units, and location names.
- Use separate validated standards for each parameter and region.